Comprendre les commandes d'annulation de Git

Objectif — Distinguer clairement le Working Directory, l'Index (Staging Area) et le Local Repo pour choisir la bonne commande d'annulation.

1. Le modèle mental : où se trouve le changement ?

Working Directory	Index (Staging Area)	Local Repo
Fichiers sur le disque : modifications pas encore préparées.	Zone de préparation : contenu sélectionné par git add pour le prochain commit.	Historique local : commits enregistrés dans la branche.

Flux simplifié : Working Directory → git add → Index → git commit → Local Repo.

Idée essentielle : une commande d'annulation agit sur une ou plusieurs de ces zones. Vérifie toujours avec git status, git diff et git diff --staged.

2. Préparer le dépôt de laboratoire

```
mkdir git-annulation
cd git-annulation
git init
git config user.name "Etudiant"
git config user.email "etudiant@example.com"

echo "Version 1" > app.txt
git add app.txt
git commit -m "Initial commit"

git status
git log --oneline
```

État de départ — Le Working Directory est propre, l'Index ne contient aucun changement en attente et le Local Repo contient le commit initial.

3. Atelier A — git restore : annuler dans le Working Directory

Situation : tu modifies un fichier suivi, mais tu n'as pas encore fait git add. Tu veux abandonner la modification.

```
echo "Code incorrect" > app.txt
git diff
git restore app.txt
cat app.txt
git status
```

Résultat attendu : app.txt contient de nouveau « Version 1 ». git restore app.txt restaure le fichier depuis l'Index.

Remarque — restaurer depuis le Local Repo

Pour imposer explicitement la version du dernier commit, utilise git restore --source=HEAD -- app.txt. Pour annuler la modification locale après git add, vois l'atelier B.

4. Atelier B — git restore --staged : retirer du Index sans perdre le fichier

Situation : tu as fait git add, mais tu veux annuler la préparation tout en conservant le contenu modifié dans le Working Directory.

```
echo "Version 2" > app.txt
git add app.txt
git status
git restore --staged app.txt
git status
git diff
```

Résultat attendu : app.txt n'est plus dans l'Index, mais sa modification est toujours présente dans le Working Directory.

À retenir — git restore --staged fichier modifie l'Index pour le réaligner sur HEAD ; il ne supprime pas la modification du Working Directory.

5. Atelier C — git rm : supprimer un fichier suivi

Situation : tu veux supprimer un fichier du Working Directory et préparer cette suppression dans l'Index.

```
echo "Configuration" > config.txt
git add config.txt
git commit -m "Ajout configuration"
git rm config.txt
git status
git commit -m "Suppression configuration"
```

Résultat attendu : git rm supprime le fichier du disque et prépare sa suppression pour le prochain commit.

Remarque— garder le fichier sur le disque

```
git rm --cached config.txt
```

Cette variante retire le fichier de l'Index et prépare sa suppression du suivi Git, mais conserve le fichier dans le Working Directory. Elle est utile, par exemple, pour arrêter de suivre un fichier qui doit rester local.

6. Atelier D — git reset fichier : annuler git add

Situation : tu as préparé un fichier, mais tu veux le sortir de l'Index sans perdre son contenu.

```
echo "Changement à conserver" > app.txt
git add app.txt
git reset app.txt
git status
git diff
```

Résultat attendu : le changement n'est plus dans l'Index, mais reste dans le Working Directory. Pour ce cas précis, git restore --staged app.txt est une alternative moderne et plus explicite.

7. Atelier E — git reset --soft, --mixed et --hard

Préparation : assure-toi que le dépôt est propre, puis crée un commit supplémentaire.

```
git restore app.txt
git add app.txt
```

```
git commit -m "Ajout version 2"
git log --oneline -3
```

Pour comparer correctement, repars du même état avant chaque variante. Après chaque essai, observe git status, git diff, git diff --staged et git log --oneline.

Commande	**Local Repo (commits)**	**Index (git add)**	**Working Directory (fichiers)**
git reset --soft HEAD~1	Annule le dernier commit	Garde les changements préparés	Garde les modifications
git reset --mixed HEAD~1	Annule le dernier commit	Retire les changements de l'Index	Garde les modifications
git reset --hard HEAD~1	Annule le dernier commit	Revient à l'état du commit précédent	Annule les modifications suivies

```
git reset --soft HEAD~1
```

Essai 1 : le commit est retiré de la branche courante, mais son contenu reste dans l'Index.

```
git reset --mixed HEAD~1
```

Essai 2 : le commit est retiré ; le contenu reste dans le Working Directory, mais n'est plus préparé. --mixed est le mode par défaut.

```
git reset --hard HEAD~1
```

Essai 3 : le commit est retiré et les changements suivis concernés dans l'Index et le Working Directory sont abandonnés.

Danger — N'utilise pas git reset --hard sur un projet réel sans vérifier l'état du dépôt et sauvegarder tout changement utile.

8. Atelier F — git revert : annuler un commit en gardant l'historique

Situation : un commit contient un bug et tu veux inverser ses effets sans réécrire l'historique partagé.

```
echo "Fonctionnalite correcte" > feature.txt
git add feature.txt
git commit -m "Ajout fonctionnalite"
```

```
echo "Bug ajoute" >> feature.txt
git add feature.txt
git commit -m "Ajout bug"
git log --oneline
git revert HEAD
git log --oneline
cat feature.txt
```

9. Atelier G — git commit --amend : corriger le dernier commit

Situation : tu as oublié un fichier dans le dernier commit local et tu veux l'y ajouter.

```
echo "Fichier A" > a.txt
git add a.txt
git commit -m "Ajout fichier A"

echo "Fichier B" > b.txt
git add b.txt
git commit --amend --no-edit
git show --stat HEAD
```

10. Atelier H — git reflog : retrouver une référence après reset

````
### Atelier H — Retrouver un commit avec `git reflog`

```bash
# 1. Créer le premier commit
echo "Bonjour" > app.txt
git add app.txt
git commit -m "Commit 1"

# 2. Créer le deuxième commit
echo "Bienvenue" >> app.txt
git add app.txt
git commit -m "Commit 2"

# 3. Annuler le dernier commit par erreur
````
````

git reset --hard HEAD~1
Le commit 2 disparaît de l'historique normal de la branche.

4. Retrouver le commit 2
git reflog
Repère la ligne "commit: Commit 2" et copie son hash.
Exemple de hash : a1b2c3d (le tien sera différent).

5. Récupérer le commit sur une nouvelle branche
git branch recuperation a1b2c3d
Remplace a1b2c3d par le hash trouvé dans git reflog.
Cette commande crée une branche qui pointe vers le commit retrouvé.

6. Aller sur la branche de récupération
git switch recuperation

Résultat : tu retrouves le commit 2 et son contenu.
```

À retenir :

`git reflog` permet de retrouver un ancien commit après un reset.
`git branch` permet de le récupérer sur une nouvelle branche.

**Exemple :**

### Atelier H — Récupérer un fichier HTML avec `git reflog`

```bash
# 1. Créer un dépôt Git
mkdir site-test
cd site-test
git init

# 2. Créer une page HTML

```
echo '<h1>Bienvenue</h1>' > index.html
git add index.html
git commit -m "Creation page HTML"
# Le premier commit contient : <h1>Bienvenue</h1>

# 3. Modifier la page HTML
echo '<h1>Bienvenue sur mon site</h1>' > index.html
git add index.html
git commit -m "Modification page HTML"
# Le deuxième commit contient : <h1>Bienvenue sur mon site</h1>

# 4. Vérifier l'historique
git log --oneline
# Tu vois les deux commits.

# 5. Annuler le dernier commit par erreur
git reset --hard HEAD~1
# Le deuxième commit disparaît de l'historique de la branche.
# Le fichier index.html contient maintenant : <h1>Bienvenue</h1>

# 6. Retrouver le commit supprimé de l'historique de la branche
git reflog
# Repère la ligne "commit: Modification page HTML".
# Copie le hash de ce commit.

# 7. Créer une branche pour récupérer ce commit
git branch recuperation HASH_DU_COMMIT
# Remplace HASH_DU_COMMIT par le vrai hash trouvé avec git reflog.

# 8. Aller sur la branche de récupération
git switch recuperation

# 9. Vérifier le contenu retrouvé
cat index.html
```

Résultat attendu :

<h1>Bienvenue sur mon site</h1>

Le fichier HTML et le commit sont récupérés sur la branche recuperation.

Conclusion :

- `git reset --hard` a ramené le fichier HTML à son ancienne version.
- `git reflog` permet de retrouver le commit précédent, puis une nouvelle branche permet de retrouver la version HTML modifiée.

11. Autres commandes d'annulation utiles

Besoin	Commande	Effet principal
Restaurer un fichier depuis un commit donné	git restore --source=<commit> -- fichier	Change le Working Directory pour ce fichier ; ne déplace pas HEAD.
Annuler un commit de merge	git revert -m 1 <commit>	Crée un commit inverse en choisissant le parent principal ; à manier avec précaution.
Prévisualiser les fichiers non suivis supprimables	git clean -n	Simulation uniquement ; liste ce qui serait supprimé.
Supprimer les fichiers non suivis	git clean -f	Suppression réelle et potentiellement irréversible ; vérifier d'abord avec -n.
Ancienne commande pour restaurer un fichier	git checkout <commit> -- fichier	Syntaxe historique ; préférer git restore pour clarifier l'intention.
Changer de branche	git switch <branche>	Ne sert pas à annuler un fichier ; change de branche.

12. Tableau de décision rapide

Je veux…	Commande recommandée
Abandonner une modification non préparée	git restore fichier
Retirer un fichier de l'Index mais garder son contenu local	git restore --staged fichier
Supprimer un fichier suivi du disque et de	git rm fichier

l'Index	
Annuler le dernier commit local en gardant les changements préparés	git reset --soft HEAD~1
Annuler le dernier commit local en gardant les changements non préparés	git reset --mixed HEAD~1
Abandonner les changements suivis locaux et reculer la branche	git reset --hard <commit> — dangereux
Annuler les effets d'un commit publié	git revert <commit>
Corriger le dernier commit local	git commit --amend
Retrouver un commit après un reset	git reflog

13. Mini-évaluation — choisis la commande

1. Tu as modifié app.txt sans faire git add et tu veux abandonner la modification.

Réponse : git restore app.txt

2. Tu as fait git add et tu veux garder le fichier modifié mais le retirer de l'Index.

Réponse : git restore --staged app.txt (ou git reset app.txt)

3. Un commit déjà partagé doit être annulé sans supprimer ce commit de l'historique.

Réponse : git revert <commit>

4. Tu veux annuler le dernier commit local tout en gardant les changements préparés.

Réponse : git reset --soft HEAD~1

5. Tu veux supprimer un fichier suivi et préparer cette suppression.

Réponse : git rm fichier

Règle d'or

Avant d'annuler —

1) git status

2) git diff

3) git diff --staged

4) git log --oneline.

Identifie si le changement se trouve dans le Working Directory, l'Index ou le Local Repo. Préfère git revert pour annuler un commit partagé et évite git reset --hard si tu dois conserver des changements.